



Diagnostic Tooling:

How to get insights from your .NET services hosted in Kubernetes containers?

Dr.-Ing. Tom Crecelius

For Feedback:



Diagnostic Tooling: How to get insights from your .NET services hosted in Kubernetes containers?



Outline

Diagnostic Artifacts, Concepts, Tools

Problem introduction

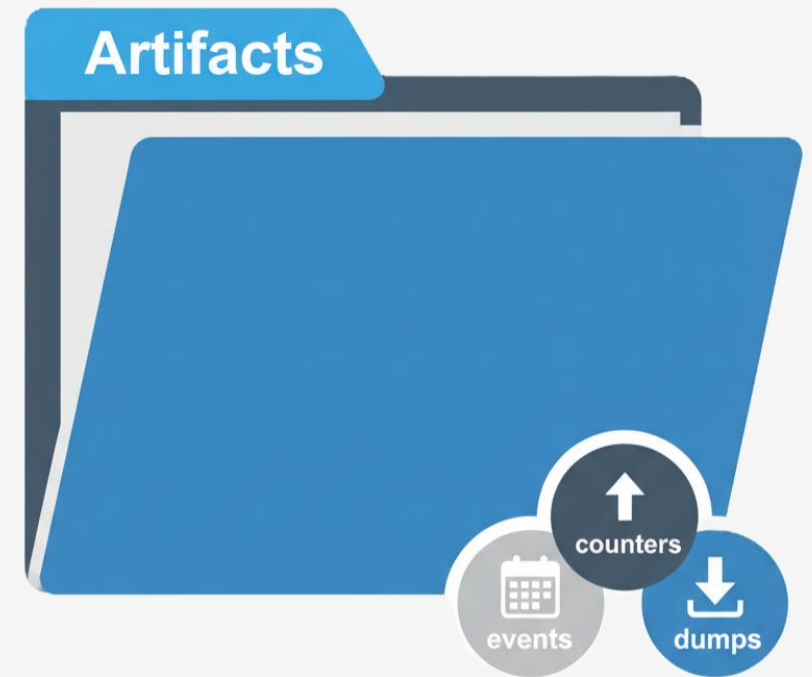
Walk-through of Kubernetes setups

- Sidecar Configuration
- Ephemeral Containers

Diagnostic Artifacts, Concepts, Tools

Diagnostic Artifacts

- What are Diagnostic Artifacts?
 - Files and captured data
 - represent application state
 - Used for analyzing runtime characteristics
- When are Diagnostic Artifacts needed - always
 - Problem diagnostics
 - Efficiency evaluation
 - Performance analysis



.NET Diagnostic Fundamentals

Events

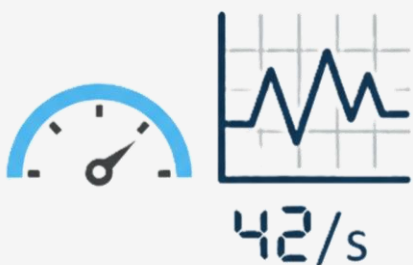


- A single diagnostic record
- Signal of what happened and where
- Trace: a time ordered collection



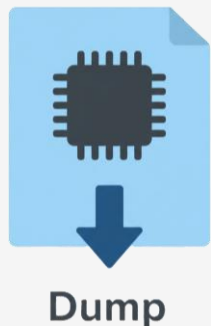
- Emitted by .NET runtime (since .NET Core 3.0)

EventCounters



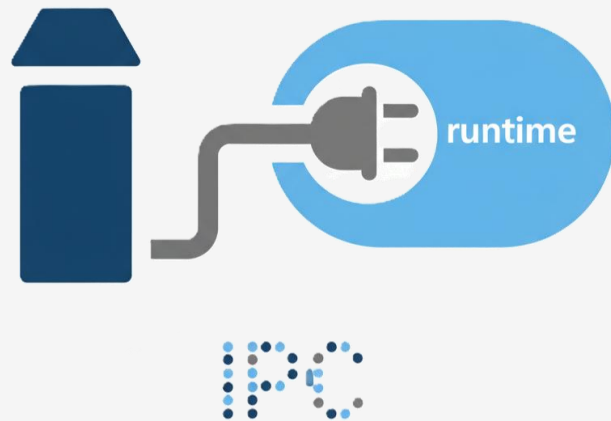
- Sampling the current process state
- Metrics: Observable “numbers”

Dumps



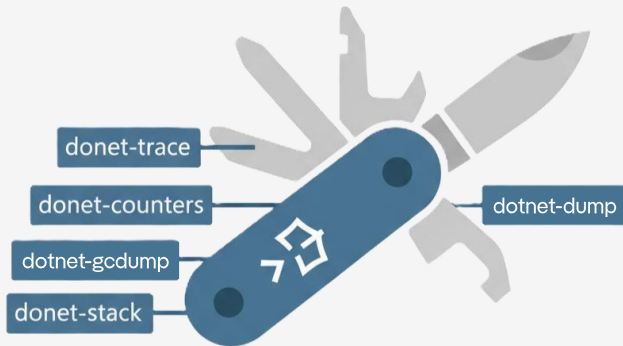
- A raw snapshot of the process memory
- Full dump, Mini dump, Heap dump
- .NET specific: Heap Snapshot (aka. GC dump)

EventPipes & Diagnostic Ports



- Event Pipes:
 - mechanism emitting Events and EventCounters (i.e., “data source”)
 - Used to create managed heap snapshots of live processes
- Diagnostic Ports:
 - two-way communication channel (i.e., “the connection point”)
 - Sending commands, receiving events, counters, dumps
 - by default (Linux runtime): `/tmp/dotnet-diagnostic-<pid>-<unique-digits>-socket`
 - → Can be changed by configuration
- Multiplatform: Windows/Linux/macOS for cross-platform diagnostics

Tools



dotnet-trace

- Collects runtime traces for perf. analysis, incl. GC activity and async behavior

dotnet-counters

- Monitors live runtime counters such as CPU, memory, or ASP.NET request metrics

dotnet-dump

- Captures and analyzes process dumps to investigate crashes or memory leaks

dotnet-gcdump

- Captures lightweight GC heap snapshots to analyze managed memory usage without a full process dump

dotnet-stack

- Captures managed thread stack traces to diagnose hangs, deadlocks, thread starvation, or blocking code.

Introduction

CID GmbH



Dr.-Ing. Tom Crecelius

Technology Evangelist –
Software Analysis and Diagnostics

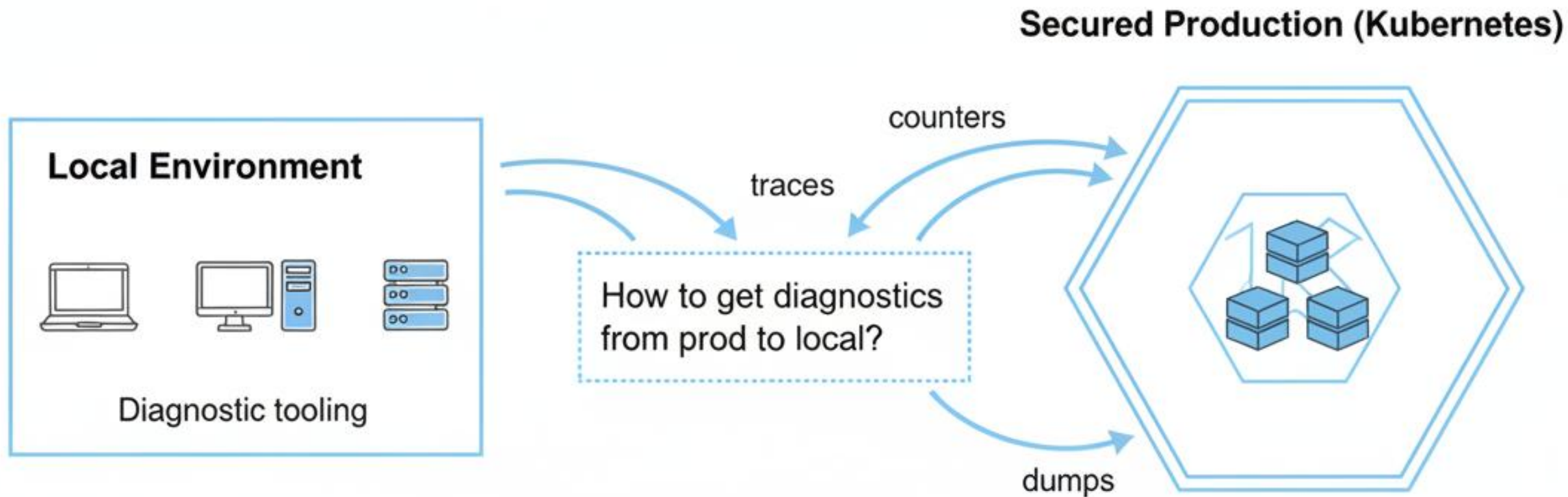
☎ +49 6051 8846-0 ✉ t.crecelius@cid.com

[in linkedin.com/in/tom-crecelius](https://www.linkedin.com/in/tom-crecelius)

To Connect:



Problem introduction



Walk-through of Kubernetes setups

Store Diagnostic tools in your app's container

- in Dockerfile:

```
1 dotnet tool install --tool-path /tools dotnet-gcdump
```

- Download pre-build binary for your architecture

```
1 curl -L -o dotnet-gcdump \
2 https://aka.ms/dotnet-gcdump/linux-x64
```

- Copy pre-build binary into the App's Container

```
1 kubectl cp dotnet-gcdump \
2 wrd2026-diagnostics/demo-memory-leak-cdc5cbbf5-rt48s:/app/dotnet-gcdump
```

- Bad Idea!

- Weakening Security
- Mixing of Concerns
- Not possible in a production setup

- Exec into App's Container & change permissions

```
1 kubectl exec -n "wrd2026-diagnostics" \
2 -it "demo-memory-leak-cdc5cbbf5-rt48s" \
3 -- bash
4
5 chmod u+x /app/dotnet-gcdump
```



Simple demo-memory-leak App

See Example-Manifests on GitHub:

<https://tinyurl.com/DiagnosticTooling>



```
1 apiVersion: apps/v1
2 kind: Deployment
3 metadata:
4   name: demo-memory-leak
5   namespace: wrd2026-diagnostics
6   labels:
7     app: demo-memory-leak
8 spec:
9   replicas: 1
10  selector:
11    matchLabels:
12      app: demo-memory-leak
13  template:
14    metadata:
15      labels:
16        app: demo-memory-leak
17    spec:
18      imagePullSecrets:
19        - name: <registry-secrets>
20
21      containers:
22        - name: demo-memory-leak
23          image: <your-registry.net>/demo-memory-leak:0.1.0
24          imagePullPolicy: IfNotPresent
25
26      ports:
27        - containerPort: 8080
28      env:
29        - name: ASPNETCORE_URLS
30          value: http://0.0.0.0:8080
31
32      resources:
33        requests:
34          cpu: "100m"
35          memory: "256Mi"
36        limits:
37          cpu: "500m"
38          memory: "1Gi"
```

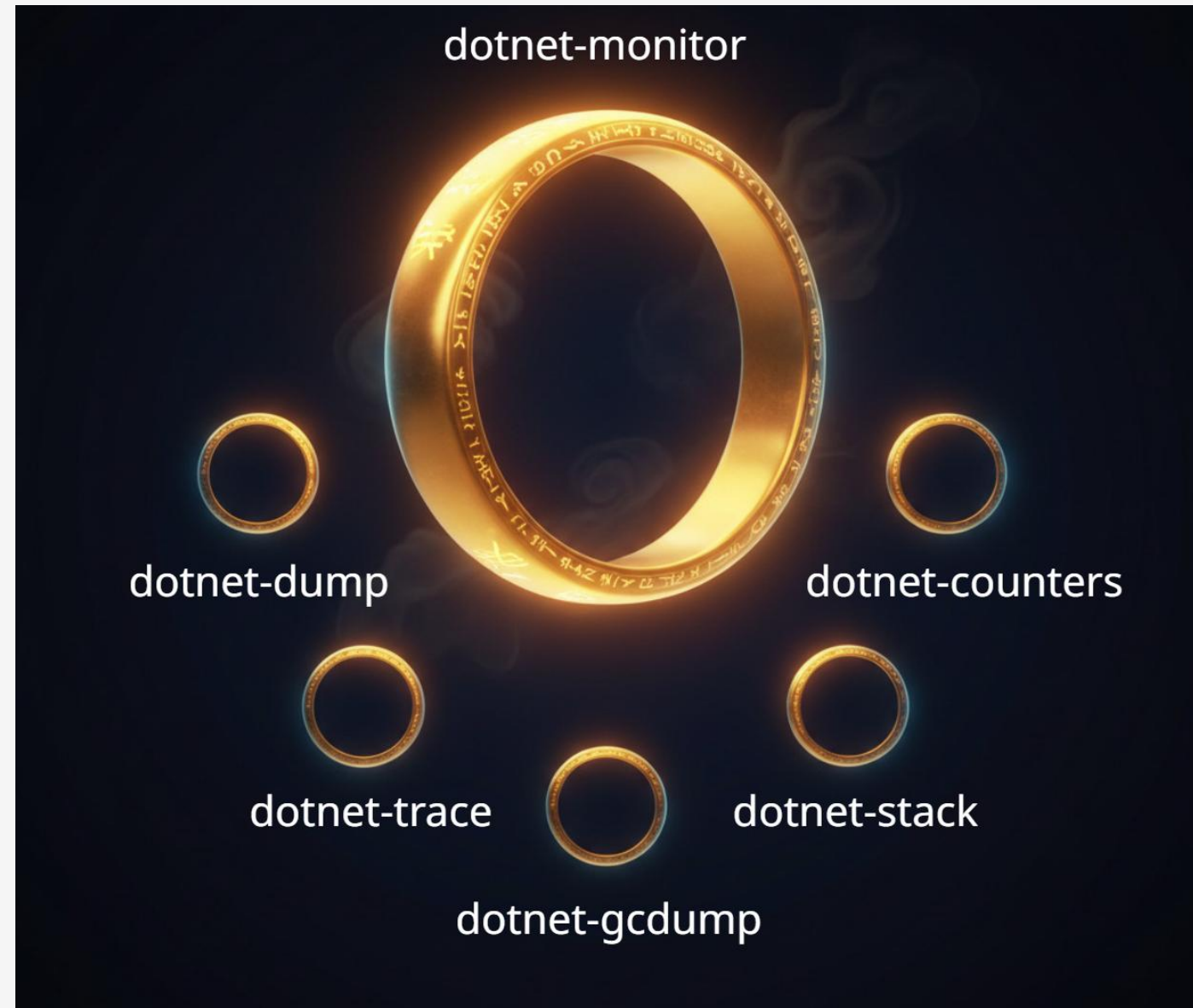
Production-Ready App & Deployment Manifest

- Chiseled images (no shell)
- Run as user
- Restricted privileges
- Read-only filesystem

```
1  +-- 14 lines: apiVersion: apps/v1.....
15     labels:
16         app: demo-memory-leak
17     spec:
18         imagePullSecrets:
19             - name: <registry-secrets>
20
21         # Pod-level security defaults
22         securityContext:
23             runAsNonRoot: true
24             runAsUser: 1654
25             runAsGroup: 1654
26
27         containers:
28             - name: demo-memory-leak
29               image: <your-registry.net>/demo-memory-leak:0.1.0
30               imagePullPolicy: IfNotPresent
31
32               securityContext:
33                 seccompProfile:
34                     type: RuntimeDefault
35                 allowPrivilegeEscalation: false
36                 readOnlyRootFilesystem: true
37                 capabilities:
38                     drop:
39                         - ALL
40
41               ports:
42                 - containerPort: 8080
43               env:
44                 - name: ASPNETCORE_URLS
45                   value: http://0.0.0.0:8080
46               resources:
47  +-- 6 lines: requests:.....
```

dotnet-monitor

- One tool to rule them all
- [Microsoft](#): ".. to monitor .NET applications in production environments .."
- Diagnostics:
 - On-Demand: HTTP API
 - Always-on: Rule-based triggers



Sidecar: dotnet-monitor

```
1  +-- 16 lines: apiVersion: apps/v1.....
17  spec:
18    imagePullSecrets:
19      - name: <registry-secrets>
20
21    # Pod-level security defaults
22    securityContext:
23      fsGroup: 2000
24      runAsNonRoot: true
25      runAsUser: 1654
26      runAsGroup: 1654
27
28    containers:
29      - name: demo-memory-leak
30  +-- 11 lines: image: <your-registry.net>/demo-memory-leak:0.1.0
41
42      ports:
43        - containerPort: 8080
44      env:
45        - name: ASPNETCORE_URLS
46          value: http://0.0.0.0:8080
47      volumeMounts:
48        - name: tmp-volume
49          mountPath: /tmp
50
51      resources:
52        requests:
53          cpu: "100m"
54          memory: "256Mi"
55        limits:
56          cpu: "500m"
57          memory: "1Gi"
58
59      - name: dotnet-monitor
60        image: mcr.microsoft.com/dotnet/monitor:10
```

```
58      - name: dotnet-monitor
59        image: mcr.microsoft.com/dotnet/monitor:10
60        args: ["collect", "--no-auth"]
61        imagePullPolicy: Always
62        securityContext:
63          seccompProfile:
64            type: RuntimeDefault
65        ports:
66          - name: dotnet-monitor
67            containerPort: 52323
68        env:
69          - name: DOTNETMONITOR_Urls
70            value: http://localhost:52323
71        volumeMounts:
72          - name: tmp-volume
73            mountPath: /tmp
74        resources:
75          requests:
76            cpu: 50m
77            memory: 32Mi
78          limits:
79            cpu: 250m
80            memory: 256Mi
81
82        volumes:
83          - name: tmp-volume
84            emptyDir: {}
```

Configure a dedicated Diagnostic Port

```
28 containers:
29   - name: demo-memory-leak
30     image: <your-registry.net>/demo-memory-leak:0.1.0
31     imagePullPolicy: IfNotPresent
32
33   securityContext:
34     seccompProfile:
35       type: RuntimeDefault
36     allowPrivilegeEscalation: false
37     readOnlyRootFilesystem: true
38     capabilities:
39       drop:
40         - ALL
41
42   ports:
43     - containerPort: 8080
44   env:
45     - name: ASPNETCORE_URLS
46       value: http://0.0.0.0:8080
47     - name: DOTNET_DiagnosticPorts
48       value: "/diagnosticArtifacts/dotnet-mymonitor.sock,nosuspend"
49   volumeMounts:
50     - name: artifacts-volume
51       mountPath: /diagnosticArtifacts
52   resources:
53     requests:
54       cpu: "100m"
55       memory: "256Mi"
56     limits:
57       cpu: "500m"
58       memory: "1Gi"
```

```
60   - name: dotnet-monitor
61     image: mcr.microsoft.com/dotnet/monitor:10
62     args: ["collect", "--no-auth"]
63     imagePullPolicy: Always
64     securityContext:
65       seccompProfile:
66         type: RuntimeDefault
67     ports:
68       - name: dotnet-monitor
69         containerPort: 52323
70     env:
71       - name: DOTNETMONITOR_Urls
72         value: http://localhost:52323
73       - name: DOTNETMONITOR_Storage__DefaultSharedPath
74         value: /diagnosticArtifacts
75       - name: DOTNETMONITOR_DiagnosticPort__ConnectionMode
76         value: "listen"
77       - name: DOTNETMONITOR_DiagnosticPort__EndpointName
78         value: /diagnosticArtifacts/dotnet-mymonitor.sock
79     volumeMounts:
80       - name: artifacts-volume
81         mountPath: /diagnosticArtifacts
82     resources:
83       requests:
84         cpu: 50m
85         memory: 32Mi
86       limits:
87         cpu: 250m
88         memory: 256Mi
89
90     volumes:
91       - name: artifacts-volume
92         emptyDir: {}
```

Access dotnet-monitor API

```
1 kubectl -n "wrd2026-diagnostics" \  
2 port-forward "demo-memory-leak-cdc5cbbf5-rt48s" 52323:52323
```

```
1 curl http://localhost:52323/ps  
2 curl http://localhost:52323/gcdump?pid=1 -o leak.gcdump
```



Secure dotnet-monitor API

- Add authentication
 - Microsoft Entra ID
 - API Key
- Enable TLS Encryption
- Mount Kubernetes Secrets in
`/etc/dotnet-monitor`

```
1 > dotnet-monitor generatekey
2
3 Authorization: Bearer eyJhbGciOiJFUzM4NCIsI...
4
5 Settings in Json format:
6 {
7   "Authentication": {
8     "MonitorApiKey": {
9       "PublicKey": "eyJjcniOiJQLTM4NCIsI...",
10      "Subject": "82e14544-..."
11    }
12  }
13 }
```

```
1 apiVersion: v1
2 kind: Secret
3 metadata:
4   name: dotnet-monitor-secrets
5 type: Opaque
6 stringData:
7   Authentication__MonitorApiKey__PublicKey: >-
8   | <the-generated-api-key>
9   Authentication__MonitorApiKey__Subject: "<the-generated-subject>"
10  Kestrel__Certificates__Default__Path: "/etc/ssl/certs/keystore.p12"
11  Kestrel__Certificates__Default__Password: "<the-self-chosen-cert-password>"
```

```
1 curl -k -H "Authorization: Bearer eyJjcniOiJQLTM4NCIsI..." \
2 "https://localhost:52323/gcdump?pid=1" -o mydump.gcdump
```

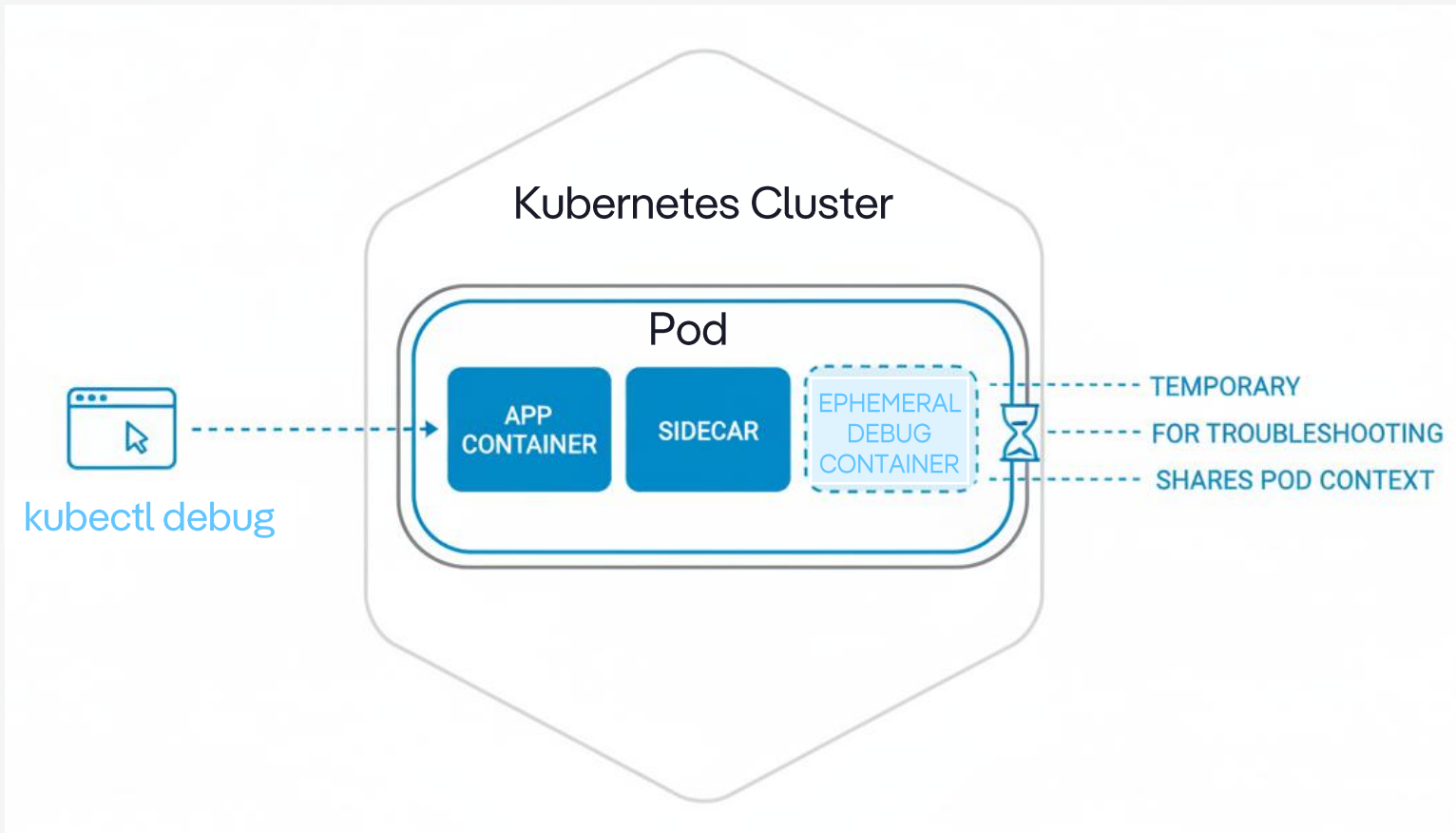
Configuring dotnet-monitor

- Configure EgressProviders
 - Filesystem
 - Azure Blob Storage
 - S3
- Use --no-http-egress
- Mount Kubernetes ConfigMap in `/etc/dotnet-monitor`

```
1 apiVersion: v1
2 kind: ConfigMap
3 metadata:
4   name: dotnet-monitor-settings
5   namespace: wrd2026-diagnostics
6 data:
7   settings.json: |
8     {
9       "$schema": "https://aka.ms/dotnet-monitor-schema",
10      "Egress": {
11        "FileSystem": {
12          "monitorFile": {
13            "directoryPath": "/diagnosticsEgressArtifacts/ready",
14            "intermediateDirectoryPath": "/diagnosticsEgressArtifacts/tmp"
15          }
16        },
17        "S3Storage": {
18          "monitorS3Blob": {
19            "endpoint": "http://localhost:9000",
20            "bucketName": "myS3Bucket",
21            "accessKeyId": "minioUser",
22            "secretAccessKey": "mySecretPassword",
23            "regionName": "us-east-1",
24            "copyBufferSize": 1024
25          }
26        }
27      }
28    }
```

Ephemeral Containers

- Added after pod creation
- Cannot be restarted, updated, or removed
- stale metadata (→ POD restart)
- Feature limited
 - No resource requests/limits
 - Can only mount existing volumes
 - No probes, no port exposure, etc.



```
1 kubectl debug -n "wr2026-diagnostics" \  
2 "demo-memory-leak-cdc5cbbf5-rt48s" \  
3 --image=hub.docker.com/library/busybox:1.38 \  
4 --target=demo-memory-leak \  
5 --container=ephemeral-busybox \  
6 -- sh -c 'sleep 365d' # --> keeps live for 365 \  
7                       # even after exiting the shell
```



Ephemeral Diagnostics Container

- “Kubectl debug”, can’t be used due to
 - volume mounts,
 - securityContext
 - etc.

```
1 kubectl replace --raw \  
2 /api/v1/namespaces/wrd2026-diagnostics/pods/demo-memor  
3 -f ephemeralContainer.json
```

```
1 kubectl exec -n "wrd2026-diagnostics" \  
2 -it "demo-memory-leak-cdc5cbbf5-rt48s" \  
3 -c "diag-debug-live" -- bash
```

```
1 {  
2   kind: Pod,  
3   apiVersion: v1,  
4   metadata: {  
5     name: demo-memory-leak-cdc5cbbf5-rt48s,  
6     namespace: wrd2026-diagnostics  
7   },  
8   spec: {  
9     ephemeralContainers: [  
10      {  
11        name: diag-debug-live,  
12        image: your-registry.net/demo-diagnostic-tooling:0.1.0,  
13        command: [  
14          sh,  
15          -c,  
16          sleep 365d  
17        ],  
18        securityContext: {  
19          seccompProfile: {  
20            type: RuntimeDefault  
21          },  
22          allowPrivilegeEscalation: false  
23        },  
24        stdin: true,  
25        tty: true,  
26        resources: {},  
27        terminationMessagePath: /dev/termination-log,  
28        terminationMessagePolicy: File,  
29        imagePullPolicy: IfNotPresent,  
30        targetContainerName: demo-memory-leak,  
31        volumeMounts: [  
32          {  
33            name: tmp-volume,  
34            mountPath: /tmp  
35          },  
36          {  
37            name: artifacts-volume,  
38            mountPath: /diagnosticsArtifacts  
39          }  
40        ]  
41      }  
42    ]  
43  }  
44 }
```



Ephemeral dotnet-monitor Container

```
1 {
2   apiVersion: v1,
3   kind: Pod,
4   metadata: {
5     name: "",
6     namespace: wrd2026-diagnostics
7   },
8   spec: {
9     ephemeralContainers: [
10      {
11        name: ephemeral-dotnet-monitor,
12        image: mcr.microsoft.com/dotnet/monitor:10,
13        targetContainerName: demo-memory-leak,
14        stdin: true,
15        tty: true,
16        command: [
17          dotnet-monitor,
18          collect,
19          --no-auth
20        ],
21        securityContext: {
22          seccompProfile: {
23            type: RuntimeDefault
24          },
25          allowPrivilegeEscalation: false
26        },
27        env: [
28          {
29            name: DOTNETMONITOR_DiagnosticPort__ConnectionMode,
30            value: Listen
31          },
32          {
33            name: DOTNETMONITOR_DiagnosticPort__EndpointName,
34            value: /diagnosticArtifacts/dotnet-diag.sock
35          },
36          {
37            name: DOTNETMONITOR_URLS,
38            value: http://0.0.0.0:52323
39          }
40        ],
41        volumeMounts: [
42          {
43            name: artifacts-volume,
44            mountPath: /diagnosticArtifacts
45          }
46        ],
47        resources: {},
48        terminationMessagePath: /dev/termination-log,
49        terminationMessagePolicy: File,
50        imagePullPolicy: IfNotPresent
51      }
52    ]
53  }
54 }
```

```
1 {
2   kind: Pod,
3   apiVersion: v1,
4   metadata: {
5     name: demo-memory-leak-cdc5cbbf5-rt48s,
6     namespace: wrd2026-diagnostics
7   },
8   spec: {
9     ephemeralContainers: [
10      {
11        name: ephemeral-dotnet-monitor,
12        image: mcr.microsoft.com/dotnet/monitor:10,
13        targetContainerName: demo-memory-leak,
14        stdin: true,
15        tty: true,
16        command: [
17          dotnet-monitor,
18          collect,
19          --no-http-egress
20        ],
21        securityContext: {
22          seccompProfile: {
23            type: RuntimeDefault
24          },
25          allowPrivilegeEscalation: false
26        },
27        env: [
28          {
29            name: DOTNETMONITOR_Storage__DefaultSharedPath,
30            value: /diagnosticsArtifacts
31          },
32          {
33            name: DOTNETMONITOR_DiagnosticPort__ConnectionMode,
34            value: Listen
35          },
36          {
37            name: DOTNETMONITOR_DiagnosticPort__EndpointName,
38            value: /diagnosticsArtifacts/dotnet-mymonitor.sock
39          }
40        ],
41        volumeMounts: [
42          {
43            name: artifacts-volume,
44            mountPath: /diagnosticArtifacts
45          },
46          {
47            name: dotnet-monitor-volume,
48            mountPath: /etc/dotnet-monitor
49          },
50          {
51            name: dotnet-monitor-certificate-volume,
52            mountPath: /etc/ssl/certs/dotnet-monitor-tls
53          },
54          {
55            name: egress-volume,
56            mountPath: /diagnosticsEgressArtifacts
57          }
58        ],
59        resources: {},
60        terminationMessagePath: /dev/termination-log,
61        terminationMessagePolicy: File,
62        imagePullPolicy: IfNotPresent
63      }
64    ]
65  }
66 }
```

Summary

Diagnostic Tools

- Always needed
- Do not include them to production images
- Do not allow to download or copy them into production containers

Dotnet-Monitor (recommended)

- Configure it as sidecar with security in mind
- Use Triggers & HTTP API
- Use EgressProviders for convenience & hardening
- Reserves some, but not much CPU/Memory resources

Ephemeral Containers

- Limited but can be used for diagnostics
- Still recommended to use dotnet-monitor
- Cannot be removed, updated replaced, or restarted.

Thank you for your attention!

To Connect:



For Feedback:



Diagnostic Tooling: How to get insights from your .NET services hosted in Kubernetes containers?

Dr.-Ing. Tom Crecelius

Technology Evangelist – Software Analysis and Diagnostics